Ultra-Low-Latency Non-LLM Retrieval-Augmented Generation: Architectures, Algorithms, and Engineering Techniques for Sub-200ms Execution at Scale

The requirement to perform Retrieval-Augmented Generation (RAG) over enterprise-scale datasets containing tens of millions to billions of documents under a strict 200-millisecond pipeline SLA eliminates standard Large Language Model (LLM) architectures from the serving path[1]. In traditional RAG implementations, autoregressive generative decoding alone introduces 500 milliseconds to several seconds of Time-To-First-Token (TTFT) overhead, creating an insurmountable bottleneck for latency-critical applications such as high-frequency trading context extraction, real-time agent memory, and live voice interactions[2]. To achieve deterministic, highly relevant contextual responses within sub-200ms operational windows, enterprise architectures deploy Non-LLM RAG systems[1].

These Non-LLM systems substitute generative autoregressive decoders with high-speed neural sentence embedders, multi-vector late interaction scoring, graph-based eigenvector centrality algorithms, singular value decomposition (SVD) matrix compression, and symbolic knowledge graph slot-filling engines[5]. Operating over massive vector indexes and document corpora requires an integrated system design spanning high-dimensional vector quantization, asynchronous CPU-GPU task orchestration, hardware-level model compilation, and sub-linear candidate search pruning[2].

To satisfy a strict sub-200ms p99 SLA across a massive corpus, every processing phase within the pipeline is bound to microsecond- and millisecond-level execution budgets. The end-to-end processing pipeline partitions its execution budget across seven discrete, highly optimized processing stages.

Pipeline Execution Stage	Underlying Architecture / Technology Stack	Target Latency (p50)	Target Latency (p99)
1. Request Ingestion & Proxy	HTTP/2 / Rust Tokio / Actix-web single binary[1]	2 ms	5 ms

2. Query Encoding & Normalization	TensorRT / ONNX Runtime Nano Embedding Engine[2]	8 ms	15 ms
3. Vector Index Search	HNSW / IVF-PQ / SQ8 (Milvus, Qdrant, Vespa)[2]	10 ms	25 ms
4. Sparse-Dense Fusion (RRF)	SIMD-accelerated rank score fusion & deduplication[2]	1 ms	3 ms
5. Contextual Reranking	INT8 TensorRT Cross-Encoder / PLAID Engine[2]	25 ms	50 ms
6. Non-LLM Context Synthesis	SVD Matrix Decomposition / TextRank / Symbolic Slot-Filling[8]	15 ms	45 ms
7. Serialization & Delivery	Binary FlatBuffers / Server-Sent Events (SSE)[2]	2 ms	5 ms
End-to-End Pipeline Total	**Fully Compiled Asynchronous Non-LLM Pipeline**	**63 ms**	**148 ms**

Architectural Paradigms for Non-LLM Retrieval and Context Synthesis

SVD-Based Hierarchical Extractive RAG (SVD-RAG)

Hierarchical document indexing structures, such as RAPTOR, recursively cluster document passages and build tree representations to enable multi-granular retrieval across high-level concepts and fine-grained details[5]. However, conventional implementations rely on generative LLMs to synthesize abstractive summaries for every cluster node in the hierarchy, creating severe build-time and query-time computational overhead[5]. SVD-RAG addresses this bottleneck by replacing generative abstractive summarization with deterministic, local Singular Value Decomposition (SVD) applied directly to dense sentence embedding matrices[5].

During tree construction and context extraction, a cluster of N retrieved sentences $\{s_1, s_2, \ldots, s_N\}$ is mapped into an embedding matrix $M \in \mathbb{R}^{N \times D}$, where D represents the embedding dimension[14]. SVD-RAG computes an economy-sized matrix decomposition formulated as:

$$M = U\Sigma V^T$$

In this matrix formulation, $U \in \mathbb{R}^{N \times r}$, $\Sigma \in \mathbb{R}^{r \times r}$, and $V \in \mathbb{R}^{D \times r}$, where rank $r = \min(N, D)$ [14]. The singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0$ along the diagonal matrix Σ quantify the semantic energy distributed across the principal components of the document cluster[14]. Rather than selecting a fixed number of sentences, the engine dynamically selects the minimum number of principal components k required to satisfy a target cumulative energy ratio threshold τ (typically set to $\tau = 0.95$):

$$E(k) = \frac{\sum_{i-1}^{k} \sigma_i}{\sum_{i-1}^{r} \sigma_i} \geq \tau$$

Once k is established, each sentence i receives an informational salience score determined by its projection energy across the dominant k principal components:

$$\text{score}(i) = \sum_{j=1}^{k} \sigma_j^2 \cdot U_{i,j}^2$$

The system extracts the k highest-scoring sentences and sequences them according to their original document positions to preserve structural readability[14]. Because SVD operates as a local matrix operation without external network dependencies, tree construction completes up to 317 times faster than LLM summarization (0.1 seconds versus 31.7 seconds) while retaining retrieval accuracy within 1% to 5% of abstractive baselines and incurring zero generative token costs[5].

Multi-Vector Late-Interaction Engines (ColBERTv2 with PLAID)

Standard single-vector embedding architectures compress multi-page passages into a single dense vector, which frequently leads to semantic loss and poor retrieval performance on fine-grained token queries[7]. Conversely, full cross-encoder models process entire query-document pairs through attention layers, which achieves high accuracy but is too

computationally expensive for initial candidate generation across massive datasets[9].
Multi-vector late-interaction architectures, such as ColBERTv2, preserve token-level representations for both queries and documents, calculating context relevance via late token interactions using the MaxSim operator[7].

For a query q containing token embeddings E_q and a candidate passage d containing token embeddings E_d, the overall late-interaction relevance score is defined as:

$$S(q,d) = \sum_{i \in |q|} \max_{j \in |d|} \left(E_{q,i} \cdot E_{d,j}^T \right)$$

Executing MaxSim calculations over millions of multi-vector document matrices within sub-50ms constraints requires specialized search engine infrastructure, such as the Performance-optimized Late Interaction for Asymmetric Search (PLAID) engine[11]. PLAID compresses individual token embeddings using residual vector quantization down to 2-bit or 4-bit representations and constructs an Auxiliary Navigation Table (ANT) over clustered token centroids[7].
During execution, PLAID applies a pruned multi-stage retrieval process. First, it evaluates query tokens against pre-clustered centroid maps to prune up to 95% of non-relevant document candidates[7]. Second, it selectively decompresses residual vectors only for the surviving top candidate passages[12]. Third, it computes the final MaxSim score using SIMD hardware vector instructions[7]. This late-interaction paradigm delivers the precision of token-level alignment at searching scales exceeding billions of vectors, completing candidate retrieval in under 40 milliseconds[11].

Eigenvector Centrality Graph Extraction (LexRank / TextRank)

For real-time context extraction where document context must be condensed into concise text without utilizing generative decoders, non-LLM systems deploy graph-based eigenvector centrality algorithms[13]. Once a high-speed retrieval pass returns a candidate set of passage chunks, the text is segmented into distinct sentence nodes $S = \{s_1, s_2, \ldots, s_n\}$ to build a fully connected lexical graph[20].

The system builds an adjacency matrix W, where each edge weight W_{ij} measures the inter-sentence similarity between sentence s_i and sentence s_j [20]. In modern low-latency pipelines, similarity is computed using cosine distance over light sentence embeddings or continuous TF-IDF vectors:

$$W_{ij} = \frac{s_i \cdot s_j}{\|s_i\| \|s_j\|}$$

To handle variation in sentence connectivity, continuous LexRank and modified TextRank

engines calculate centrality by running a power iteration algorithm over the stochastic matrix to solve for the stationary probability distribution vector $\mathbf{p}$:

$$\mathbf{p} = \left(\frac{1-d}{N}\mathbf{E} + d\mathbf{W}^T\mathbf{D}^{-1} \right) \mathbf{p}$$

In this formulation, d represents the damping factor (typically configured to 0.85), $\mathbf{E}$ is a square matrix filled with ones, and $\mathbf{D}$ is the diagonal degree matrix containing row sums of W[13]. The power iteration loop converges rapidly, typically requiring fewer than 20 iterations to compute stationary probabilities across candidate pools of hundreds of sentences[13]. The sentences corresponding to the highest values in vector $\mathbf{p}$ are extracted and concatenated in logical order to form a factual summary[19]. Operating entirely in local CPU memory, this process executes in 5 to 15 milliseconds[13].

Hybrid Dense-Sparse RRF Retrieval with Localized Rescoring

To balance semantic search with exact keyword matching—such as retrieving specific technical codes, invoice numbers, or proper names—high-throughput pipelines implement parallel hybrid retrieval[2]. The query is issued concurrently to a dense vector index via an HNSW graph search and a sparse inverted index using BM25 scoring[2]. Because dense cosine distances and sparse BM25 scores exist on different numerical scales, standard linear combination requires continuous score calibration[2]. Non-LLM systems bypass score normalization by applying Reciprocal Rank Fusion (RRF) to merge candidate lists based on ordinal rank[2]:

$$RRF(d \in D) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

Here, M represents the set of retrieval channels (dense and sparse), $r_m(d)$ is the ordinal rank of document d within channel m, and k is a smoothing constant typically set to 60[2]. RRF execution takes under 2 milliseconds using SIMD-accelerated array sorting[2].

The top 20 candidate passages produced by RRF are passed to a local cross-encoder model (such as Jina Reranker v3) running on a hardware-accelerated runtime[2]. The cross-encoder processes the query and each candidate passage as a single concatenated sequence, computing full cross-attention interactions to score context relevance[2]. Truncating the final candidate pool down to the top 3 rescored passages improves precision by 2.9 percentage points and reduces downstream context noise, completing the entire hybrid retrieval and reranking pass in under 35 milliseconds[2].

Symbolic Knowledge Graph Entity Linking and Template Hydration

In enterprise domains subject to regulatory compliance—such as healthcare protocols, legal compliance, and financial auditing—nondeterministic text synthesis can introduce operational risk[1]. Non-LLM architectures address this by combining dense vector retrieval with symbolic Knowledge Graph (KG) query generation and template slot-filling[6].

Incoming user queries undergo high-speed named entity recognition (NER) and dependency parsing to extract entity nodes and relational predicates[8]. The system uses these extracted terms to perform 1-hop and 2-hop sub-graph traversals over an in-memory graph store, isolating verified factual triples[8]. Rather than passing these sub-graph triples to an autoregressive model for natural language generation, the pipeline routes the graph facts into symbolic evaluation engines, such as Prolog logic solvers or pre-compiled JSON document templates[6]. The system hydra-populates structured fields directly from validated database attributes, returning structured payloads with zero hallucination risk and completing execution in under 20 milliseconds[6].

High-Efficiency Engineering Techniques for Sub-200ms Pipeline Execution

Vector Index Hyperparameter Tuning and High-Ratio Quantization

Scaling non-LLM RAG to datasets containing hundreds of millions to billions of vectors requires balancing memory footprint against query latency and search recall[4]. Operating uncompressed 1024-dimensional Float32 vectors in flat memory indexes requires 4 gigabytes of RAM per million vectors, which quickly exhausts memory hardware limits and causes latency spikes due to cache misses[9]. High-efficiency vector stores deploy approximate nearest neighbor (ANN) graph indexes paired with scalar or product quantization[4].

In Hierarchical Navigable Small World (HNSW) graph indexes, performance relies on three primary configuration parameters: node link degree M, build candidate depth ef_construction, and runtime search candidate depth ef_search[9]. For general enterprise text retrieval, setting M = 16 to 32 and ef_construction = 128 creates a sparse small-world graph that supports logarithmic search trajectories[9]. At query time, ef_search serves as the primary runtime lever[9]. Restricting ef_search to a range of 40 to 80 yields greater than 95% recall-at-10 while keeping search latencies between 5 and 12 milliseconds[9]. Increasing ef_search beyond 200 provides marginal recall gains but increases search iterations linearly, causing tail latencies to exceed 100 milliseconds[9].

At billion-vector scales, memory management shifts from HNSW to Inverted File (IVF) indexes combined with Product Quantization (PQ) or 8-bit Scalar Quantization (SQ8)[4]. Scalar Quantization compresses 32-bit floating-point values into 8-bit integers, cutting memory footprint by 75% while maintaining over 98% search recall[9]. Product Quantization breaks D -dimensional vectors into m distinct sub-vectors, mapping each sub-vector to its closest cluster centroid index across 2^k pre-computed codebooks[9]. Quantizing a 1024-dimensional Float32

vector with configuration $m = 32, k = 8$ reduces storage requirements from 4,096 bytes down to 32 bytes—a 128x compression factor[9]. Configuring the IVF centroid count to nlist = 4 * sqrt(N) and probing nprobe = 8 to 16 candidate cells per query restricts vector distance scans to localized regions of the index, maintaining sub-30ms search performance across multi-billion-vector corpora[9].

Index Type & Quantization	Memory Compression Ratio	Search Recall (Recall@10)	Latency Profile (100M+ Scale)	Primary Operational Trade-off
HNSW (Uncompressed Float32)	1x Baseline (4 KB / vector)[9]	99.1%	5 – 12 ms[9]	High RAM cost; unviable for billion-vector datasets[9]
HNSW + SQ8 (Scalar Quantization)	4x Reduction (1 KB / vector)[9]	98.2%	8 – 15 ms[9]	Minor recall loss; preserves graph traversal speed[9]
IVF-SQ8 (Inverted File + SQ8)	4x to 6x Reduction[9]	95.5%	12 – 22 ms[9]	Requires periodic index rebuilding as data drifts[9]
IVF-PQ (PQ32, k=8)	128x Reduction (32 B / vector)[9]	91.4%	18 – 35 ms[9]	Distance scores estimated via codebook lookups[9]

Asynchronous Heterogeneous Orchestration and Operator Coalescing

Pipeline bottlenecks often stem from thread starvation and head-of-line blocking when mixing CPU-bound operations—such as tokenization, BM25 scoring, and graph parsing—with GPU-bound operations like dense embedding and cross-encoder inference[6]. Modern low-latency execution frameworks (such as Halo) resolve these bottlenecks using asynchronous event-driven task schedulers[6].

Execution workflows are parsed into Directed Acyclic Graphs (DAGs) of discrete task nodes

managed by an asynchronous coordinator[6]. The scheduler evaluates candidate task nodes dynamically using a depth-first priority strategy[6]. Ready CPU tasks are prioritized based on their topological proximity to downstream GPU dependencies[6]. Executing CPU preparation steps just-in-time ensures GPU inference queues remain continuously saturated, preventing execution stalls and GPU idling[6].

Under high concurrent request volumes, RAG systems frequently receive duplicate or overlapping retrieval requests[6]. Schedulers implement operator signature coalescing to reduce redundant computation[6]. Incoming tasks generate a normalized signature derived from their operator type and input arguments[6]. When concurrent threads issue identical operational signatures, the engine merges those requests into a single physical execution call[6]. Upon execution completion, the output results are fan-out distributed to all waiting request listeners, reducing redundant index scans and embedding computations by up to 40% under bursty traffic[6].

Model Compilation and Hardware Quantization for Cross-Encoders

While cross-encoder reranking models improve retrieval relevance, running unoptimized PyTorch models introduces latency overhead that can consume the majority of a 200ms pipeline budget[2]. High-efficiency systems accelerate cross-encoder models using framework compilation and integer precision quantization[2].

PyTorch cross-encoder models (such as Jina Reranker or BGE-Reranker) are exported to Open Neural Network Exchange (ONNX) formats and compiled into optimized TensorRT execution engines[2]. TensorRT applies graph-level optimizations, fusing multi-head attention layers, removing redundant transpose operations, and selecting auto-tuned CUDA kernels specific to the host GPU architecture[6].

Compiling cross-encoder models with INT8 Post-Training Quantization (PTQ) or Quantization-Aware Training (QAT) converts 32-bit floating-point weights into 8-bit integer representations[6]. This quantization allows the model to utilize specialized hardware instructions (such as NVIDIA Tensor Core DP4A SIMD operations), yielding a 3x to 5x inference speedup over FP32 runtimes while maintaining reranking accuracy within 0.5% of unquantized baselines[6]. Combining INT8 quantization with dynamic sequence truncation (restricting cross-encoder evaluation to the top 15-20 RRF candidates) reduces reranking latency down to

20-30 milliseconds[2].

Unified Compiled Runtimes and Zero-Copy I/O

Garbage collection pauses, dynamic memory allocation overhead, and inter-process communication (IPC) latencies in high-level interpreted environments can consume up to 100 milliseconds of runtime budget[2]. Sub-200ms RAG pipelines address this by implementing unified single-binary runtimes written in compiled languages such as Rust or C++[2].
Engineered using asynchronous event-loop runtimes (such as Tokio with Actix-web or Axum), compiled microservices handle concurrent socket connections with microsecond-level routing overhead[2]. Internal data structures avoid string copy operations and dynamic allocations by utilizing zero-copy memory layouts, such as Apache Arrow or FlatBuffers, for IPC between vector indices, rerankers, and client sockets[15]. Persistent HTTP/2 connection pooling eliminates TCP handshake and TLS negotiation overhead on incoming requests[1]. Server-Sent Events (SSE) stream output data back to clients incrementally as processing stages finish, ensuring low initial response latencies[2].

Multi-Tiered Memory Caching and Pre-Computed Centroid Filtering

System latency can be minimized by avoiding redundant compute operations across multi-stage pipelines through structured caching strategies[3].
Low-latency pipelines implement a multi-tiered caching architecture[3]. Tier 1 (L1 GPU Memory Cache) stores high-frequency query embeddings and cross-encoder outputs directly in GPU VRAM, returning identical or near-identical queries in under 1 millisecond[3]. Tier 2 (L2 Host CPU Memory Cache) stores pre-tokenized text segments, sparse BM25 term matrices, and extracted sub-graphs in lock-free shared memory structures, resolving warm requests in under 5 milliseconds[3].
For late-interaction multi-vector search (ColBERTv2), computing dense token-to-token inner product matrices across thousands of candidate documents introduces computational overhead[7]. Pipelines eliminate redundant matrix multiplication by constructing an Auxiliary Navigation Table (ANT) during batch index ingestion[7]. The ANT pre-computes interaction matrices between token cluster centroids[7]. At query time, the retrieval engine scores query tokens against these pre-computed centroid maps first, pruning up to 95% of non-relevant document matrices prior to token decompression[7]. This pre-filtering step reduces multi-vector late-interaction evaluation times down to sub-40ms windows[11].

Technical Comparison of Non-LLM RAG Paradigms

Selecting an appropriate Non-LLM RAG architecture depends on dataset scale, latency constraints, domain precision requirements, and infrastructure resources.

Non-LLM RAG Paradig	Core Search & Extractio	Algorithmic Mechani	Scale Ceiling	Execution	Accuracy & Fidelity	Enterprise Deploym

m	n Engines	sm		Latency	Profile	ent Fit
SVD-Based Extractive RAG	Vector Store + SVD Linear Algebra Engine[5]	Singular Value Decomposition ($M =$) over sentence embedding matrices[5]	Tens of Millions of Chunks[5]	**15 – 45 ms** [cite: 5, 14]	High (Extracts key verbatim source sentences)[5]	Hierarchical document summarization, contract analysis[2]
Multi-Vector Late-Interaction	PLAID Engine / ColBERT v2 Index[11]	Token-level MaxSim interaction across residual quantized vectors[7]	Billions of Vectors[4]	**20 – 50 ms** [cite: 7, 11, 12]	Very High (Preserves token alignment and nuance)[7]	Code search, technical documentation, multi-lingual RAG[1]
Eigenvector Centrality (LexRank)	Hybrid Retriever + Graph Centrality Engine[13]	Power iteration solving stationary distribution over similarity graphs[13]	Millions of Documents[9]	**5 – 15 ms** [cite: 13]	Medium-High (Ranks central informative sentences)[13]	News summarization, live feed distillation[13]
Hybrid RRF + Cross-Encoder	Dual HNSW/BM25 + INT8 TensorRT Engine[2]	Sparse-dense Reciprocal Rank Fusion + cross-attention reranking[2]	Billions of Vectors[4]	**25 – 60 ms** [cite: 2]	Very High (Combines semantic and exact keyword match)[2]	Legal compliance, medical search, enterprise knowledge bases[2]

Symbolic KG Slot-Filling	Graph Store + Symbolic Rule Engine[8]	NER entity linking, N-hop graph traversal, template hydration [8]	Hundreds of Millions of Entities[24]	**10 – 30 ms** [cite: 8, 22]	Absolute (Zero hallucination risk; deterministic matching) [8]	Financial compliance auditing, medical triage[22]

Conclusions and System Integration Outlook

Achieving sub-200ms end-to-end processing SLAs over enterprise datasets containing millions to billions of records requires moving beyond standard LLM-centric RAG designs[1]. Autoregressive language generation introduces decoding latency and resource demands that conflict with low-latency operational budgets[2]. Non-LLM RAG architectures resolve this trade-off by replacing text generation with deterministic context extraction and synthesis mechanisms—including SVD matrix energy scoring, multi-vector late-interaction MaxSim evaluation, eigenvector centrality graph extraction, and symbolic knowledge graph slot-filling[5].
Maximizing the performance of these architectural paradigms requires pairing them with hardware-aligned system engineering techniques[2]. By combining HNSW vector index hyperparameter tuning and product quantization with asynchronous heterogeneous scheduling, INT8 cross-encoder compilation, single-binary compiled runtimes, and multi-tiered RAM/VRAM caching, systems can consistently execute context retrieval, ranking, and extraction within 60 to 150 milliseconds at p99 tail latencies[2].
As enterprise datasets continue to grow, the production path for low-latency retrieval lies in custom compiled runtimes that integrate mathematical matrix decomposition with hardware-accelerated vector search engines[5]. These Non-LLM RAG architectures deliver scalable context processing while meeting strict operational latency budgets[1].

Works cited

1. code_generation - LLMOps Database - ZenML, https://www.zenml.io/llmops-tags/code-generation
2. GitHub - RustyRAG/RustyRAG: Production-grade RAG API built in Rust. Hybrid search with HNSW dense vectors and BM25 sparse matching, cross-encoder reranking, layout-aware document extraction via Docling, and 94.5% accuracy on Open RAG Bench. Powered by Cerebras, Groq, Milvus, and Jina AI., https://github.com/AlphaCorp-AI/RustyRAG
3. Stateful Large Language Model Serving with Pensieve - arXiv, https://arxiv.org/html/2312.05516v3
4. Best Vector Databases in 2026: A Complete Comparison Guide - Firecrawl, https://www.firecrawl.dev/blog/best-vector-databases

5. svd-rag: efficient tree-organized retrieval-augmented generation - arXiv, https://arxiv.org/pdf/2607.10316
6. Batch Query Processing and Optimization for Agentic Workflows - arXiv, https://arxiv.org/html/2509.02121v2
7. Unified and Efficient Approach for Multi-Vector Similarity Search - arXiv, https://arxiv.org/html/2604.02815v1
8. Question Entity and Relation Linking to Knowledge Bases via CLOCQ - CEUR-WS.org, https://ceur-ws.org/Vol-3337/smart-paper1.pdf
9. Vector Database Deep Dive: How They Actually Work - Ajit Singh, https://singhajit.com/vector-database-deep-dive/
10. Pinecone vs Weaviate vs Qdrant vs pgvector: 2026 buyer guide | Aidoptio, https://aidoptio.com/blog/pinecone-vs-weaviate-vs-qdrant-vs-pgvector
11. Awesome Vector Databases - GitHub, https://github.com/paradoxe35/awesome-vector-databases
12. MV-IVF: Multi-Vector Retrieval via Multi-Vector Clustering - OpenReview, https://openreview.net/pdf?id=JZaw4wtJWZ
13. Extractive Summarization using Extended TextRank Algorithm - ACL Anthology, https://aclanthology.org/2024.icon-1.54.pdf
14. SVD-RAG: Efficient Tree-Organized Retrieval-Augmented Generation via Singular Value Decomposition - arXiv, https://arxiv.org/html/2607.10316v1
15. Tasks - Wiro API Docs, https://wiro.ai/docs/tasks
16. Mitigating Hallucinations in Discipline Inspection QA: A Two-Stage RAG Framework with Late Interaction and Reranking - MDPI, https://www.mdpi.com/2079-9292/15/3/541
17. Beyond Chunk-and-Embed: How LazyGraphRAG, A-RAG, and, https://buzzgrewal.medium.com/beyond-chunk-and-embed-how-lazygraphrag-a-rag-and-colqwen2-are-rewriting-retrieval-augmented-35b8c9949472
18. Neural IR-based Approaches for Bangla Text Retrieval - ResearchGate, https://www.researchgate.net/publication/395307370_Neural_IR-based_Approaches_for_Bangla_Text_Retrieval
19. Extractive Summarization of Text with the LexRank Algorithm - rdrr.io, https://rdrr.io/cran/lexRankr/man/lexRank.html
20. Text Summarization using the TextRank Algorithm (with Python) - Analytics Vidhya, https://www.analyticsvidhya.com/blog/2018/11/introduction-text-summarization-textrank-python/
21. LLMs for Cybersecurity in the Big Data Era: A Comprehensive Review of Applications, Challenges, and Future Directions - MDPI, https://www.mdpi.com/2078-2489/16/11/957
22. jyyang621/DailyArXiv: Thanks to https://github.com/zezhishao/DailyArXiv.git · GitHub - GitHub, https://github.com/jyyang621/DailyArXiv
23. Exploring Retrieval-Augmented Generation (RAG) and Its Alternatives - Medium, https://raghunaathan.medium.com/exploring-retrieval-augmented-generation-rag-and-its-alternatives-bf9e2f337f88
24. ReGraM: Region-First Knowledge Graph Reasoning for Medical Question

Answering - arXiv, https://arxiv.org/html/2601.09280v1
25. building more reliable and scalable ai systems with foundation model programming a dissertation submitted to the - Stacks, https://stacks.stanford.edu/file/zc625xj7842/okhattab_dissertation_final_v3-augmented.pdf
26. From HNSW to Information-Theoretic Binarization: Rethinking the Architecture of Scalable Vector Search - arXiv, https://arxiv.org/pdf/2601.11557
27. Together AI vs Runpod (2026): Which Is Better? - RFP.wiki, https://www.rfp.wiki/artificial-intelligence/cloud-ai-developer-services/together-ai/runpod
28. latency_optimization - LLMOps Database - ZenML, https://www.zenml.io/llmops-tags/latency-optimization
29. RoutIR: Fast Serving of Retrieval Pipelines for Retrieval-Augmented Generation - arXiv, https://arxiv.org/pdf/2601.10644
30. RAG Infrastructure: Building Production Retrieval-Augmented Generation Systems - Introl, https://introl.com/blog/rag-infrastructure-production-retrieval-augmented-generation-guide
31. fastapi - LLMOps Database - ZenML, https://www.zenml.io/llmops-tags/fastapi
32. Daily Papers - Hugging Face, https://huggingface.co/papers?q=Knowledge%20Graph%20(KG)